Algorithms Design and Analysis
DP Sheet

March - 2026

Table of Contents

FIRST: PROBLEMS and SOLUTIONS[1]

QUESTION 1: Coin Change

Suppose A ={a_1,a_2,a_3,….a_k} is a set of distinct coin values (all the a_i are positive integers) available in a particular country. The coin changing problem is defined as follows: Given integer n find the minimum number of coins from A that add up to n assuming that all coins have value in A. You should assume that a_1=1 so that it is always possible to find some set that adds up to n.Design a O(nk) dynamic programming algorithm for solving the coin changing problem, that is, given inputs A and n it outputs the minimum number of coins in A to add up to n.(It is not necessary to say what the coins are)

Answer

- Let's consider "A" array that consists of the coin values >= 1:-

10	2	5	1

- Let "C [P]" be an array that contains *minimum number* of coins of dominations needed to make change for "P" cents.
- Let "S [P]" be an array that contains index of the optimal coin (In array "A") needed to make change for "P" cents.
- By using the following recursive formula we guarantee that we calculate the *minimum number* of coins of dominations needed to make change for "P" cents:-

$$C[P] = \begin{cases} 0 & \text{If } p = 0 \\ \min_i : d_i <= p \{1 + C[p - d_i]\} & \text{If } p > 0 \end{cases}$$

- Use this algorithm to fill in the two arrays:-

```
Change (d, k, n)
        C[0] ← 0
        for p ← 1 to n
                min ← ∞
                for i ← 1 to k
                        if A[i] <= p then
                                if 1 + C[p − d[i]] < min then
                                min ← 1 + C[p − d[i]]
                                coin ← i
                C[p] ← min
                S[p] ← coin
        return C and S
```

O (n * k)

- Testing with i.e. **Change(A, 4, 8)**
- Filling Arrays:-

	0	1	2	3	4	5	6	7	8
C	0	1	1	2	2	1	2	2	3
S	0	4	2	2	2	3	3	3	2

A	0	10	2	5	1

- Use this algorithm to extract minimum coins needed for the change (*Which is not actually needed in the question*):-

```
Make-Change (S, A, n)
        while n > 0
                Print A[S[n]]
                        n ← n − A[S[n]]
```

O (n)

- Output of **Make-Change(S,A, 8)**:-

n = 8
>> 2
n = 6
>> 5
n = 1
>> 1

QUESTION 2: Knapsack Variation

(a) Write the recurrence relation and solve the 0-1 knapsack problem given that you have four items having weights of 2,3,4,5 and values of 3,4,5,6 respectively. The total weight capacity equals 5, what is the complexity of your algorithm? Is it polynomial in input size?

(b) Suppose you are given a set of n objects. The weights of the objects are w1, w2, . . . ,wn, and the values of the objects are v1, v2, . . . , vn. You are given two knapsacks each of weight capacity C. Write a recurrence relation to determine the maximum value of objects that can be placed into the two knapsacks. What will be the complexity in this case?

Answer

(a) Recurrence Relation:-

$$\text{Knap}(W, N) = \begin{cases} 0 & \text{If } N = 0 \quad \textit{No Items or space} \\ \text{Knap}(W, N-1) & \text{If } w[N] > W \quad \textit{Leave it} \\ \text{Max}\{\text{Knap}(W, N-1), \text{Knap}(W - w[N], N-1) + b[N]\} & \end{cases}$$

0-1 Knapsack Solution:-

Weight\Item	0	(1)	(2)	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	(3)	3	3	3
3	0	3	4	4	4
4	0	3	4	5	5
5	0	3	(7) ←	7 ←	7

_-_-Items-_-_

(Weight, Value)

1- (2, 3)
2- (3, 4)
3- (4, 5)
4- (5, 6)

So we take items 1 and 2, to have maximum total value of 7

Complexity:-

O (n * W) →where “n” is the number of items

Polynomial in input size?? ← **I** suppose *yes* ^o) ??

(b) Recurrence Relation:-

$$\text{Knap}(W_1, W_2, N) = \begin{cases} 0 & \text{If } N = 0 \quad \textit{No Items or space} \\ \text{Knap}(W_1, W_2, N-1) & \text{If } w[N] > W_1 \,\&\, w[N] > W_2 \quad \textit{Leave it} \\ \text{Max}\{\text{Knap}(W_1, W_2, N-1), \text{Knap}(W_1 - w[N], W_2, N-1) + b[N], \text{Knap}(W_1, W_2 - w[N], N-1) + b[N]\} & \end{cases}$$

Complexity:

O (n * W_1 * W_2) →where "n" is the number of items

QUESTION 3: Exact Sum

You are given a sequence of positive integers a_1; a_2; :::; a_n, and a positive integer B. Your goal is to determine if some subsequence of a_1; a_2; :::; a_n sums to exactly B.

Answer

D&C Pseudo Code

```
Check (A, N, B):-
      If A [N] == B Then
            Return true
      End If
      If N == 0 OR B == 0 Then
            Return false
      End If
      If A [N] < B Then
            B1 = check (A, N - 1, B - A [N])
            B2 = check (A, N - 1, B)
            Return (B1 OR B2)
      Else
            Return check (A, N - 1, B)
      End If
```

D&C Complexity:-

This algorithm uses recursion, so find its recurrence and solve it:

$T(N) = 2 \times T(N-1) + \Theta(1)$

Solve by iteration method ➔ $\Theta(2^N)$

Side Note: Example on How Overlapping Can Occur

```
A = {1, 5, 10, 20, 30, 50, 100}, B = 80
```

We can reach to a common sub-problem: "**`exactlySum(3, 30)`**", which try to find combination that sum to 30 using first 3 numbers, by two ways:

1. Use 50 and bypass 30 & 20
2. Bypass 50 and use 30 & 20

Both problems need to solve **`exactlySum(3, 30)`**

Dynamic Programming Solution:-

Extra storage: 2D Boolean array of size N × B,

```
For I = 0 to N
      For J = 0 to B
            If A [I] == J Then
                  Check[I, J] = true
            Else if I == 0 Then
                  Check[I, J] = false
            Else if A [I] < J Then
                  B1 = Check[I - 1, J - A[I]]
                  B2 = Check[I - 1, J]
                  Check[I, J] = (B1 OR B2)
            Else
                  Check[I, J] = Check[I - 1, J]
            End If
      End For
End For
```

DP Complexity: Θ(N × B)

SECOND: UNSOLVED PROBLEMS

QUESTION 1: N Stairs

Suppose we are walking up n stairs. At each step, you can go up 1 stair, 2 stairs or 3 stairs. Our goal is to compute how many different ways there are to get to the top (level n) starting at level 0. We can't go down and we want to stop exactly at level n.

1. Design an efficient algorithm to solve this problem?
2. What's the complexity of your algorithm?

QUESTION 2: Super Thief

You are an amazing super thief. You have scoped out a circle of ***n***, houses that you would like to rob tonight. However, as the super thief that you are, you've done your homework and realized that you want to **avoid robbing two adjacent houses** because this will minimize the chance that you get caught. You've also scouted out the hood, so for every house ***i***, you know that the net worth you will gain from robbing that house is v_i. Thus, You want to try to **maximize the amount of money** that you can rob.

1. Explain why this problem is a good candidate for a DP algorithm?
2. Give a DP algorithm to find the maximum amount of money that you can steal given that you never rob two houses that are adjacent? State whether you are using the bottom up approach or the top down approach?
3. Modify your algorithm so that it also returns the best set of houses to rob instead of the maximum value you can steal?

QUESTION 3: Subset Sum

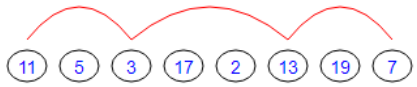
In computer science, the **subset sum problem** is an important problem in complexity theory and cryptography. Given a set (or multiset) of integers, is there a non-empty subset whose sum is zero?

For example, given the set {−7, −3, −2, 5, 8}, the answer is *yes* because the subset {−3, −2, 5} sums to zero.

1. Write a recurrence relation?
2. Write a pseudocode of a DP solution for this problem?
3. What will be the complexity of the solution?

QUESTION 4: Rabbit Crossing a River

The Rabbit is going to cross the river. There is a straight chain of tiny isles across the flow and the animal should jump from one to another. At each of the isles there are some candies. When the Rabbit arrives to the new isle, it collects all the candies here.
However, the Rabbit could not jump directly to the next isle in the chain - it just is too strong to make short jumps. So, instead, it can jump over the one or two isles (i.e. from the 1st for example to 3rd or 4th but not to 2nd or 5th and further). Also the Rabbit could not jump back.



The amount of 34 candies. Obviously he could do better if the path is chosen more wisely.
Your task is to choose the best path for Rabbit over the given chain of isles - i.e. to maximize the amount of the candies collected. Note that Rabbit starts from 1st isle and finishes either on the Nth or (N-1)th where N is the total number of isles (because from these two it will necessarily jump immediately to the other bank).

Examples

Input	Output
11 5 3 17 2 13 19 7	48
9 7 12 7 16 3 7 17 14 13 4 6 11 6 3 3 5 4 11 3 15 12 14 2 15 19 11 12	157

QUESTION 5: Ali Baba in a Cave

Ali Baba and his thieves enter a cave that contains a set of *n* **types of items** from which they are to select some number of items to be theft. Each item type has a *weight*, a *profit* and a *number of available instances*. Their objective is to choose the set of items that fits the possible load of their Camels and maximizes the profit. Note the following:

1- Each item should be **taken as a whole** (i.e. they can't take part of it)
2- They can take the same item **one or more time** (according to its number of instances).

Given ***n* triples** where each triple represents the (**weight, profit, number of instances**) of an **item type** and the **Camels possible load**, find maximum profit that can be loaded on the Camels by the **OPTIMAL WAY**.

Complexity

Your algorithm should take **polynomial time**

Example

N = 4 Load = 10

Weight	Profit	# instances
2	1	2
4	8	2
3	6	2
4	5	2

Max Profit = 20$, as follows:

1. one instance from item2 (profit 8$)
2. two instances from item3 (profit 6$ × 2 = 12$)

THIRD: ONLINE PROBLEMS

1. **Diving for Gold @ UVa**
2. **Vacations @codeforces**
3. **Dividing coins @ UVa**

FOURTH: Tracing Questions

QUESTION1: 0-1 Knapsack

In a 0-1 Knapsack problem, it's required to fill a knapsack of weight **W** with the maximum total cost from **N** items. Each item has only one instance with weight w[i] and cost c[i]. Following is an example:

Knapsack weight (W) = 5; Items (weight, cost): (1,2), (2,3), (3, 4), (4, 5).

Answer: maximum total cost is 7. This is achieved by taking 2nd and 3rd item.

Given a knapsack of weight **W = 3**, and the following **4 items**, **fill-up** the DP solution table and **answer** the questions:

item index (i)	**1**	**2**	**3**	**4**
weight (w[i])	1	2	4	1

cost (c[i])	30	20	80	30

1. What's the final total cost in this case? What's the remaining weight in the knapsack (if any)?
2. From the filled DP table, find the optimal total cost in each of the following cases:

(knapsack weight, items count)	(0,0)	(0,3)	(3,0)	(2,1)	(3,2)	(3,3)	(2,4)
Optimal total cost	0						

Answer

1. **60, 1**
2.

(knapsack weight, items count)	(0,0)	(0,3)	(3,0)	(2,1)	(3,2)	(3,3)	(2,4)
Optimal total cost	0	**0**	**0**	**30**	**50**	**50**	**60**